

5th International Conference on Machine Learning and Data Engineering (ICMLDE 2026)

Resource-Efficient Software Vulnerability Detection Using Fine-Tuned Code Large Language Models

Rohan Babbar^a, Sariga Saji^a, Vaasanth Korupolu^a, M S Basanth^a, G Gopakumar^{a,*}

^a*Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri, India*

Abstract

Software vulnerability detection is critical for secure software development, yet traditional static analysis tools often struggle with false positives and poor generalization. This paper presents a transformer-based framework for CWE classification on the Juliet C/C++ Test Suite. A key contribution is a template-aware dataset splitting strategy that prevents structural data leakage between training and testing samples, enabling more reliable evaluation than conventional random splitting. Weighted cross-entropy loss addresses class imbalance during training. Multiple transformer architectures including RoBERTa, CodeBERT, and CodeT5 are comparatively evaluated for CWE classification. Experimental results show that fine-tuned transformer models achieve strong vulnerability classification performance, with the proposed CodeT5-small model achieving a macro-F1 score of 0.9605 under template-aware evaluation. Ablation analysis confirms that random dataset splitting artificially inflates performance compared to template-aware evaluation. The findings demonstrate that compact transformer models provide an efficient and practical solution for automated vulnerability detection in secure software development workflows.

© 2026 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the International Conference on Machine Learning and Data Engineering.

Keywords: CodeT5; CWE Classification; Vulnerability Detection; Transformers; Software Security

1. Introduction

The rapid growth and deployment of large, interconnected software systems has left less time to conduct manual security reviews. As a consequence, many vulnerabilities will reach production systems alongside the initial software. Automated processes for detecting software weaknesses are now essential to the secure development life cycle.

Static Application Security Testing (SAST) tools are popular because they can analyze source code without the need to execute it. However, as they rely mainly on rules and signatures, these tools generalize poorly to new weaknesses and depend on previously known vulnerability patterns. They also generate many false positives, making developers reluctant to integrate them into agile development workflows.

Prior work at Amrita Vishwa Vidyapeetham has addressed AI-driven security analysis across multiple fronts, including source-code vulnerability detection [12, 14], CVE severity prediction [13], malware and intrusion detection [15, 19, 20], dependency-risk monitoring [16], smart-contract security assessment [17], and adversarial attack classification [18]. Across these studies, a shared challenge emerges: learned models are sensitive to evaluation methodology, motivating a more principled, leakage-aware approach to CWE benchmark evaluation.

Transformer-based models have recently demonstrated strong performance in source code understanding and vulnerability detection tasks due to their ability to capture contextual relationships within code sequences. However, ensuring reliable evaluation remains a significant challenge, particularly when benchmark datasets contain structurally similar template-generated samples that may introduce structural data leakage between training and testing sets.

This work investigates whether compact transformer architectures can provide an effective balance between vulnerability classification performance and computational efficiency for CWE detection. To improve evaluation reliability, we introduce a template-aware dataset splitting strategy that prevents structural leakage between training and testing samples. Multiple transformer-based encoder models are evaluated for CWE classification across 118 vulnerability categories using the Juliet benchmark dataset.

This paper makes the following contributions:

- We propose a template-aware dataset splitting strategy that prevents structural data leakage and enables more reliable evaluation for CWE vulnerability classification.
- We perform a comparative evaluation of transformer-based encoder models including RoBERTa, CodeBERT, CodeT5-base, and CodeT5-small for multi-class CWE classification across 118 vulnerability categories.
- We demonstrate the practical viability of compact transformer-based vulnerability detection through performance, efficiency, and generalization analysis.

2. Related Work

Detection of vulnerabilities has evolved from being appended to the analysis phase as static rules, to being modeled through code tokens, code graphs, and most recently, with transformer encoders. Rule-based and symbolic approaches were useful for established patterns. However, they were not applicable to new weaknesses and would generate a high number of false positives. In VulDeePecker [5], reframing the task as supervised learning demonstrated that neural models could learn the signals of vulnerabilities from code gadgets rather than relying only on manual rules.

Subsequent studies aimed to work with richer program representations. Sequence embeddings improved over hand-crafted features, but they still struggled to capture long-range dependencies and deeper context within source code. The introduction of transformers such as CodeBERT [1], GraphCodeBERT [2], CodeT5 [3], and CodeT5+ [4] addressed this by using self-attention to model broader program context, with strong results across vulnerability detection, defect prediction, and related code-intelligence tasks. Empirical generalizability analyses such as [8] confirm these gains, but they also make the evaluation problem more visible: large models can fit benchmark artifacts, and their cost is not trivial.

Graph-based models remain relevant where structure matters. Devign [6] learns from program semantics and data-flow relationships, showing that token-only models are not always sufficient. Several recent surveys chart the shift toward LLM-based detection: Dolcetti and Iotti [9] review large language models for code verification from both model and task perspectives; Shaon and Akter [10] catalogue machine-learning, deep-learning, and LLM methods for vulnerability detection; and Xuan et al. [11] quantify the accuracy gains that LLMs provide. A recurring theme is that, while these models push accuracy higher, they still face high false-positive rates, poor cross-project generalization, opaque predictions, dataset imbalance, and heavy compute requirements.

Complementing the broader trends above, institutional work from Amrita Vishwa Vidyapeetham has examined learning-based security methods at several levels of the software stack. Detection has been applied to source-code vulnerabilities [12, 14], network intrusions [19], and malware [20, 15]. Related tasks such as CVE severity prediction from natural-language descriptions [13], dependency-chain risk monitoring [16], smart-contract vulnerability assessment [17], and graph-based adversarial attack classification [18] have also been studied. While these contributions span diverse security domains, they share a common dependence on learned feature representations—underscoring the importance of evaluation protocols that prevent benchmark artifacts, such as template overlap, from inflating reported performance.

3. Proposed Methodology

3.1. Overview of Juliet Test Suite

The Juliet Test Suite provides a benchmark for testing software security. It has structured examples of both vulnerable and non-vulnerable code, specifically in the languages C and C++. Each test case has the following components:

- A *bad* function that contains a security weakness
- A *good* function that shows an example of a correction
- A CWE label for the weakness

Each of the vulnerability types has multiple template-based variations, providing a varied code structure while maintaining the same pattern of a given security weakness.

3.2. Dataset Statistics

Table 1. Dataset statistics for the Juliet Test Suite

Metric	Value
Total Samples	179,839
Number of CWE Classes	118
Largest Class Size	~16,000
Smallest Class Size	<10
Language	C/C++

The dataset has a long-tailed distribution of the CWE categories that is highly imbalanced. The distribution of vulnerabilities reflected in the dataset poses a significant challenge to the implementation of supervised learning models.

3.3. Data Preprocessing

This phase helps ready the dataset prior to training and includes the subsequent practices:

1. Removal of white space.
2. Comments are retained to help preserve their associated meaning.
3. Code is transformed to have a normalized structure.
4. Use of the CodeT5 tokenizer with a maximum sequence of 256 tokens. Code tokens are subjected to a subword tokenization scheme.

Retention of comments is essential since these comments help convey hints of the program's meaning and may provide insight to uncover possible program vulnerabilities. Samples of code which surpass the maximum sequence are truncated. This may leave the code sample without context, but helps keep computing resources focused and the sample input uniform.

With the above processes, the model is provided with optimized input representations.

3.4. Template-Aware Dataset Splitting Strategy

Juliet creates test cases based on templates, resulting in many samples that differ only slightly and share the same coding structures for vulnerabilities. With a random split, samples that are very similar end up in both partitions, and the score will reflect the memorization of the templates, not the generalization.

We address this by splitting at the template level. All samples from a specific template will go to either the training or the testing dataset, but not both. Because of this, the partitions are structurally distinct and we can better assess how the model predicts previously unseen patterns of vulnerabilities.

3.5. Model Architecture

We have selected CodeT5-small as a model base because it balances a small parameter budget and high performance on various tasks related to the understanding of code. When passing tokenized source code, the encoder will generate contextual embeddings that will retain relationships of the elements of a given program from both a syntactical and semantical point of view. These will then be pooled and provided to a softmax layer through a linear classification head, yielding probability scores over 118 CWEs. The goal of this training will be multi-class cross-entropy loss:

$$\mathcal{L} = - \sum_{i=1}^N y_i \log(\hat{y}_i)$$

where y_i is the true class and $\hat{y}_i$ is the predicted probability of class i .
Key characteristics of CodeT5-small:

- Encoder layers: 6
- Attention heads: 8
- Hidden size: 512
- Parameters: ~60M

3.6. Loss Function Explanation

The model is trained using the multi-class cross-entropy loss function, which measures the difference between predicted probabilities and actual labels.

The objective of this loss function is to minimize the error between the predicted class distribution and the true distribution. By penalizing incorrect predictions, the model learns to assign higher probabilities to the correct classes.

To mitigate severe class imbalance, class-weighted cross-entropy loss is incorporated during training.

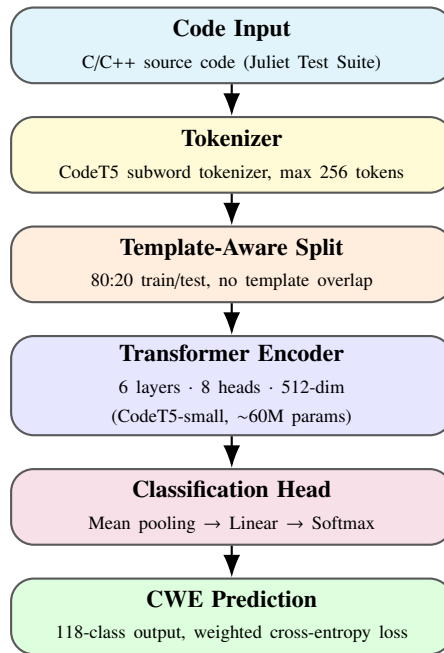


Fig. 1. Overall architecture of the proposed CWE detection framework showing dataset construction, template-aware splitting, preprocessing, CodeT5 encoding, classification, and evaluation pipeline.

We used AdamW to update the model with the following parameters:

- Batch size: 4
- Learning rate: 5×10^{-5}
- Epochs: 2
- Optimizer: AdamW

The effects of dropout and early stopping are used in the model to mitigate overfitting. The evaluation metrics of the model were accuracy, precision, recall, and the F1-score (macro). The F1-score (macro) was used to measure the overall performance of the model to evaluate all 118 categories of vulnerabilities to mitigate the effects of class imbalance.

The model was trained on a system with an NVIDIA GPU and 16GB of VRAM. Each training epoch lasted 2–3 hours. The dynamics of training and convergence will be discussed in Section 4.

4. Experimental Results

The complete version of the Juliet Test Suite is used in all experiments, and its original class distribution is maintained so as to preserve the long-tailed distribution of the benchmark. The suite covers 118 CWE categories and approximately 179,839 samples. The largest class has around 16,000 samples, and there are several classes with fewer than 10 samples.

A template-aware 80:20 split provided around 144,000 training samples, and 36,000 testing samples. Since the split is at the template level, no template sample is included in both the training and testing sets. Thus, the test set is comprised only of vulnerability structures the model has not seen during training.

4.1. Training Performance

Table 2. Training convergence across epochs

Metric	Epoch 1	Epoch 2
Training Loss	Decreasing	Stabilized
Validation Loss	Decreasing	Stabilized
Model Convergence	Rapid	Converged

4.2. Training Dynamics and Convergence Stability

To further verify training stability beyond the main two-epoch experimental setting, the CodeT5-small model was additionally trained for four epochs under the same hyperparameter configuration. This four-epoch run is reported only as a convergence-stability analysis and is not used for the main model-comparison results. Figure 2 shows the training and validation loss progression.

The training loss decreases rapidly from 0.0710 to approximately 0.0045, while the validation loss stabilizes at 0.0031 by the third epoch. The absence of divergence between training and validation losses indicates stable optimization and suggests that the model does not exhibit noticeable overfitting under the current training configuration.

The loss curves plateau after the third epoch, indicating that additional training provides minimal performance gains. These observations support the selection of a small number of training epochs for efficient model convergence.

4.3. Model Performance and Comparative Evaluation

As shown in Table 3, injection-related vulnerability categories achieve particularly high precision due to their highly structured semantic patterns. OS Command Injection (CWE-78) achieved a precision of 0.9936 and an F1-score of 0.9968 across 11,884 samples, while LDAP Injection (CWE-90) achieved a precision, recall, and F1-score of 0.9981. These results indicate that the model effectively captures the characteristic execution-based patterns associated with injection vulnerabilities.

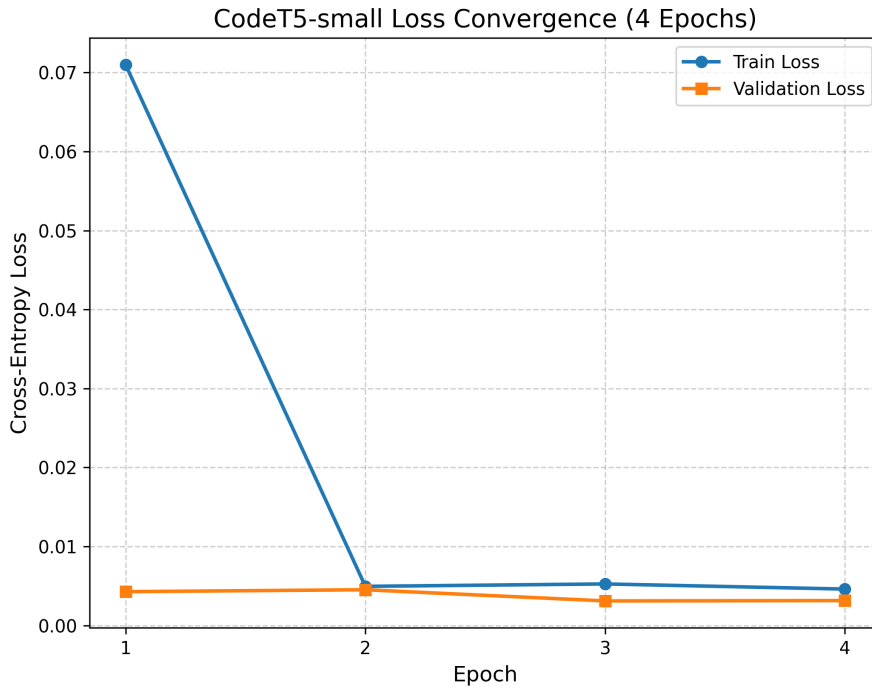


Fig. 2. Cross-entropy loss convergence of an additional CodeT5-small run over four epochs (main experimental setting uses two epochs). Both training and validation losses stabilize after the third epoch, indicating stable optimization behavior.

Table 3. Performance of representative injection vulnerability categories

CWE	Precision	Recall	F1-score
CWE-78 (OS Command Injection)	0.9936	1.0000	0.9968
CWE-90 (LDAP Injection)	0.9981	0.9981	0.9981

Table 4. Comparison of transformer-based models for CWE classification

Model	Parameters	Macro F1
CodeBERT	~125M	0.9983
RoBERTa	~125M	0.9951
CodeT5-base	~220M	0.9978
Proposed (CodeT5-small)	~60M	0.9605

The proposed CodeT5-small model achieved a macro-averaged F1-score of 96.05% under the template-aware evaluation setting, demonstrating strong balanced classification performance across multiple CWE vulnerability categories.

The results indicate that larger transformer architectures achieve higher classification performance, while the proposed CodeT5-small model significantly reduces parameter count and computational requirements. With approximately 60 million parameters, the proposed model requires substantially less memory than CodeBERT and CodeT5-base while still maintaining strong vulnerability classification performance under template-aware evaluation. This demonstrates the practical feasibility of lightweight transformer architectures for deployment-oriented secure software analysis workflows.

4.4. Impact of Training Sample Size

The number of training samples per CWE category significantly affects classification performance.

The rare categories tell the opposite story. With only a handful of examples, the model cannot separate a minority CWE from its neighbours, and misclassification rates climb. This is the direct effect of the long tail: the categories that need the most learning signal are the ones that provide the least.

4.5. Precision–Recall Analysis

Figure 3 plots class-wise precision against recall. Most categories cluster in the high-precision, high-recall region, which is expected for the well-represented CWEs where common vulnerability patterns recur often enough to be learned reliably.

The exceptions are informative. As Table 5 shows, several cryptography-related categories—CWE-325 (Missing Required Cryptographic Step), CWE-338 (Weak Pseudo-Random Number Generator), CWE-321 (Use of Hard-coded Cryptographic Key), CWE-327 (Use of Broken/Risky Cryptographic Algorithm), and CWE-328 (Reversible One-Way Hash)—score far lower. These weaknesses hinge on semantic intent rather than surface syntax, and they come with few training examples, so the model has little to generalize from. Overall, the analysis suggests that most classification errors arise from class imbalance and limited representation of minority classes rather than instability of the underlying model architecture.

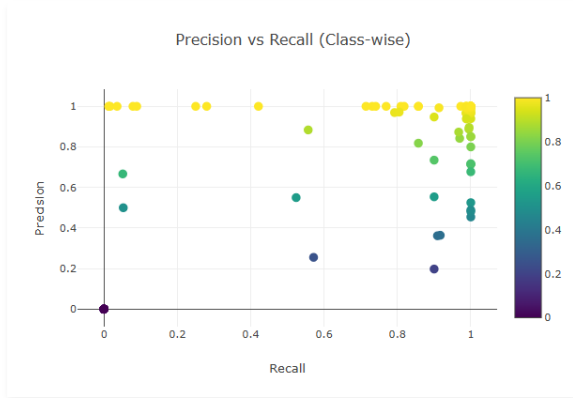


Fig. 3. Class-wise precision versus recall across CWE categories. Performance varies across classes, with underrepresented categories generally exhibiting larger variability.

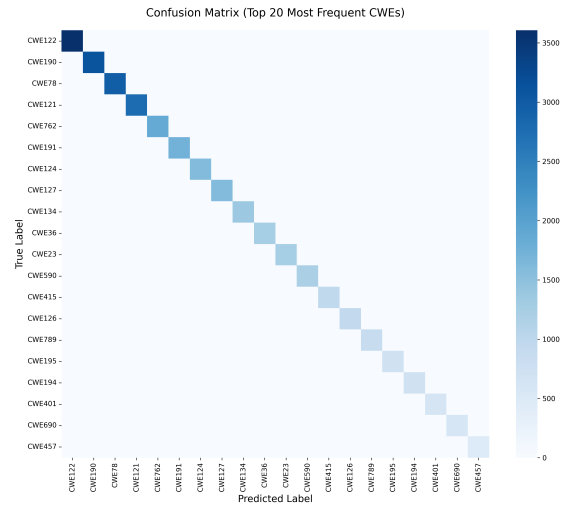


Fig. 4. Confusion matrix for the ten most frequently occurring CWE categories in the Juliet dataset. These categories were selected due to their sufficient sample representation, allowing clearer visualization of inter-class confusion patterns and common misclassification trends.

Table 5. Examples of low-performing CWE categories under template-aware evaluation. Support indicates the number of test samples for each category.

CWE	Precision	Recall	F1-score	Support
CWE-325	0.0000	0.0000	0.0000	76
CWE-338	0.0000	0.0000	0.0000	40
CWE-321	1.0000	0.0167	0.0328	300
CWE-327	1.0000	0.0789	0.1463	76
CWE-328	1.0000	0.0357	0.0690	112

4.6. Cross-Categorical Performance Variance

To evaluate performance consistency across all 118 CWE categories, we examined the class-wise F1-score distribution under template-aware evaluation. The model attains a macro-F1 score of 0.9605, while minority cryptography-related classes (Table 5) remain substantially more difficult. This indicates that overall performance is strong, but variability across underrepresented classes is still significant.

4.7. Ablation Study: Effect of Template-Aware Splitting

To assess the impact of data structure leakage, an ablation study was employed to assess the traditional random splitting reported in the literature versus the novel template-aware data partitioning reported below.

Table 6. Ablation study on dataset splitting strategy

Split Strategy	Macro F1	Template Leakage
Random Split	0.9984	96.08%
Template-Aware Split	0.9605	0.00%

Results of the ablation study are included in Table 6. Random splitting yields near perfect Macro F1 scores. However, that number is largely fictitious. By our estimate, more than 96.08% of the test set templates were also included in the training set. Hence, the model was largely rewarded for recognizing already seen data structures. In the case of the template-aware split, that overlap is removed, and the Macro F1 score drops to 0.9605.

The aim of the large gap reported in the two cases is to illustrate that the strong performance seen with random splitting is largely due to the model memorizing the training data, and to explain why template-generated benchmarks (like Juliet) can severely underestimate a detector’s real-world performance if the data split is not carefully controlled.

4.8. Pattern and Error Analysis

Pattern-based analyses suggest that different CWE classes incorporate different syntactic and structural patterns that the model is able to learn and take advantage of:

- Structs are almost always associated with memory-related vulnerabilities.
- Integer constants are associated with overflow vulnerabilities.
- Pointer arithmetic is the pattern associated with memory access errors.

Notwithstanding these patterns, the model struggles with the minority classes, where the lack of training data prevents it from learning reliable decision boundaries. For instance, the buffer overflow and improper memory access classes tend to get merged, as they both include pointer operations and boundary checks. The model gets the coarse patterns right, but fails to capture the fine-grained information needed to differentiate between the closely associated types of memory vulnerabilities.

The confusion matrix for the most dominant CWE categories reveals that the most frequent errors occur between semantically related classes of vulnerabilities.

- Majority classes dominate predictions across the data set
- Minority classes are frequently misclassified due to insufficient training data
- Some large classes are always present as predictions for misclassified minority data.

To analyze the most frequent confusions, we focused on the CWE classes that are most frequently confused. The non-random confusions are formed from memory-related coding patterns and boundary-related issues. These share similar code structure, and the model often confounds them. Because the errors are due to the proximity of the classes, they indicate an inability to distinguish closely related classes rather than a poor prediction.

Rare categories introduce a second source of errors. When a class has very few training examples, it is impossible to set a reliable boundary, and it will eventually be misclassified into other classes. This reflects the same class-imbalance effect seen in the per-class scores, now visible at the level of individual confusions.

Adding structural information to the model would likely give it the context it needs to differentiate closely related classes, especially under poor class balance.

4.9. Class Imbalance Analysis

The long tail distribution on the 118 CWE categories impacts the model in three ways:

- The major classes define the training signal.
- The minor classes lack sufficient samples.
- The model tends to predict the categories that are represented the most.

This is consistent with actual occurrence of types of software vulnerabilities, where few are common and many are rare. The rare classes do not provide the model with a signal. This is why we use class-weighted loss to train the model and evaluate it with macro-averaged metrics, where the rare classes are not overshadowed.

4.10. Computational Efficiency

With around 60 million parameters, CodeT5-small is approximately half the size of CodeBERT (125M), and a quarter of CodeT5-base (220M). Because of the small size, CodeT5-small uses significantly less memory. Models of this size can accurately classify all 118 CWE categories.

Training took approximately 2–3 hours per epoch with a batch size of 4, performed on a single GPU. CodeT5-small even has an advantage over larger models because inference takes only a few milliseconds, making the model suitable for use in an automated scanning pipeline.

5. Conclusion

The findings highlight the effectiveness of transformer-based models for software vulnerability detection. Key insights from this work are:

- Lightweight transformers offer greater flexibility and ease of use in the design of models without sacrificing performance.
- The design of a dataset, especially the method used to split the dataset, has a large effect on how reliable the evaluation of the model is and how the model behaves overall.
- Most misclassifications can largely be ascribed to class imbalance, as opposed to actual confusion of vulnerability patterns.

We have devised a transformer-based framework that uses the CodeT5-small model to classify vulnerabilities, based on the Juliet Test Suite, and located the framework's performance. Results indicate that even small models perform well on large, adequately designed, and well-structured datasets. Comparative to the conventional random splitting, template-aware splitting results in diminished data leaking and more reliable evaluations of the dataset.

The results also consider that most classification mistakes result from class imbalance and not from the genuine code patterns' closeness, which also outlines the possible of the advanced design of datasets from a machine learning perspective on code vulnerability. This method is also computationally cheaper and can be added easily.

This technique can be added to various secure software development pipelines. It can be concluded from the results of this research that small transformer models can effectively differentiate between codes and can easily locate vulnerabilities. The future can consider more effective management of the minor classes and also larger application of this technique to different real world datasets.

5.1. Limitations and Future Work

Future work will address several key areas: (1) evaluation on real-world datasets (Devign [6], Big-Vul [7], CVE repositories) to assess generalization beyond synthetic Juliet samples; (2) managing class imbalance through advanced techniques such as focal loss and few-shot learning; (3) incorporating program structure (trees, graphs) to capture semantic context and improve fine-grained distinction of related vulnerability types; and (4) combining transformers with graph-based neural networks and incorporating explainable AI to build developer trust through interpretable predictions.

Acknowledgements

The authors gratefully acknowledge Sri Mata Amritanandamayi Devi (Amma), Chancellor, Amrita Vishwa Vidyapeetham, for her inspiration and for providing financial support for the Article Processing Charges (APC) of this publication.

References

- [1] Z. Feng et al., “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020, pp. 1536–1547.
- [2] D. Guo et al., “GraphCodeBERT: Pre-training code representations with data flow,” in *Proc. International Conference on Learning Representations (ICLR)*, 2021.
- [3] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, “CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” in *Proc. 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021, pp. 8696–8708, doi: 10.18653/v1/2021.emnlp-main.685.
- [4] Y. Wang, H. Le, A. D. Gotmare, N. D. Q. Bui, J. Li, and S. C. H. Hoi, “CodeT5+: Open code large language models for code understanding and generation,” in *Proc. 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 1069–1088, doi: 10.18653/v1/2023.emnlp-main.68.
- [5] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, “VulDeePecker: A deep learning-based system for vulnerability detection,” in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2018, doi: 10.14722/ndss.2018.23158.
- [6] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, “Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [7] J. Fan, Y. Li, S. Wang, and T. N. Nguyen, “A C/C++ code vulnerability dataset with code changes and CVE summaries,” in *Proc. 17th International Conference on Mining Software Repositories (MSR)*, 2020, pp. 508–512, doi: 10.1145/3379597.3387501.
- [8] X. Zhou, D. Han, and D. Lo, “Assessing generalizability of CodeBERT,” in *Proc. 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2021, pp. 425–436, doi: 10.1109/ICSME52107.2021.00044.
- [9] G. Dolcetti and E. Iotti, “A dual perspective review on large language models and code verification,” *Frontiers in Computer Science*, vol. 7, Art. no. 1655469, 2025, doi: 10.3389/fcomp.2025.1655469.
- [10] M. S. H. Shaon and M. S. Akter, “Modern approaches to software vulnerability detection: A survey of machine learning, deep learning, and large language models,” *Electronics*, vol. 14, no. 22, Art. no. 4449, 2025, doi: 10.3390/electronics14224449.
- [11] C. D. Xuan, D. Q. Bui, and D. Q. Vu, “Large language models based vulnerability detection: How does it enhance performance?” *International Journal of Information Security*, vol. 24, Art. no. 69, 2025, doi: 10.1007/s10207-025-00983-8.
- [12] **Prabith G. S., Rohit Narayanan M., Arya A., Aneesh Nadh R., and Binu P. K., “BiT5: A bidirectional NLP approach for advanced vulnerability detection in codebase,”** *Procedia Computer Science*, vol. 233, pp. 812–821, 2024, doi: 10.1016/j.procs.2024.03.270.
- [13] **Manjunatha A., Kethan Kota, Anoop S. Babu, and Sree Vivek S., “CVE severity prediction from vulnerability description – A deep learning approach,”** *Procedia Computer Science*, vol. 235, pp. 3105–3117, 2024, doi: 10.1016/j.procs.2024.04.294.
- [14] **Aravind M., Kausal S. D., Saranya G., Sajjad Shaikat Jamal, Amer Aljaedi, and Fahad Alblehai, “Enhancing code security with CodeSheriff: A multi-language vulnerability scanner,”** *Research Square preprint*, 2024, doi: 10.21203/rs.3.rs-4757733/v1.
- [15] **S. J. Nair and S. R. Syam, “Comparing transformers and CNN approaches for malware detection: A comprehensive analysis,”** in *Proc. 2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, 2024, pp. 1–6, doi: 10.1109/ICCCNT61001.2024.10724417.
- [16] **Chinni Krishna Kowsik P., Uday Reddy H., Udhayakumar S., and Saranya G., “Software composition analysis for proactive threat detection in software dependencies with real-time security monitoring,”** in *Proc. 2025 2nd International Conference on Computing and Data Science (ICCDs)*, 2025, doi: 10.1109/ICCDs64403.2025.11209681.
- [17] **A. Shewale, D. Thallapalli, and S. Udhayakumar, “Enhancing smart contract security: A comprehensive vulnerability assessment framework,”** in *Proc. 2025 2nd International Conference on Computing and Data Science (ICCDs)*, 2025, doi: 10.1109/ICCDs64403.2025.11209716.
- [18] **G. Akshitha Sai, K. Naga Sai Likhitha, M. Pavan Kalyan, P. Anjani Devi, and C. P. Prathibhamol, “Combinational features with centrality measurements on GCN+LR classification of adversarial attacks in homogenous graphs,”** in *Proc. 2022 Fourteenth International Conference on Contemporary Computing (IC3-2022)*, New York, NY, USA: ACM, 2022, pp. 573–581, doi: 10.1145/3549206.3549305.
- [19] **R. Vinayakumar et al., “Deep learning approach for intelligent intrusion detection system,”** *IEEE Access*, vol. 7, pp. 41525–41550, 2019.
- [20] **R. Vinayakumar et al., “Robust intelligent malware detection using deep learning,”** *IEEE Access*, vol. 7, pp. 46717–46738, 2019.